MODULE 3

Advanced Patterns & Workflows

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 1 / Third**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Decompose complex, multi-file tasks into manageable sub-tasks that Claude Code can execute reliably
2. Apply the ReAct (Reason + Act) framework to guide Claude through iterative problem-solving workflows
3. Configure generation controls (temperature and thinking budget) to optimize output quality for different task types
4. Use iterative refinement strategies to transform first-draft code into production-quality output
5. Build and maintain team CLAUDE.md files that enforce consistent standards across engineering teams

SESSION TIMING BREAKDOWN

0:00–0:10	**Section 1: Problem Decomposition — Breaking Down Complex Tasks**
0:10–0:18	**Section 2: The ReAct Framework — Reason + Act**
0:18–0:26	**Section 3: Generation Controls — Temperature & Thinking Budget**
0:26–0:34	**Section 4: Iterative Refinement — From Good to Great**
0:34–0:42	**Section 5: Team CLAUDE.md — Scaling Standards Across Engineers**
0:42–0:45	**Wrap-Up & Transition to Module 4**

0:00 – 0:10	SECTION 1: Problem Decomposition — Breaking Down Complex Tasks

1. Problem Decomposition — Breaking Down Complex Tasks

1.1 Why Decomposition Matters

The biggest mistake engineers make with AI coding tools is giving them tasks that are too large. When you ask Claude Code to “build an entire REST API with authentication, database models, and tests,” it tries to do everything at once — and the result is usually mediocre across the board.

> *“The art of vibe coding is knowing how big a bite the AI can chew. Small, focused tasks produce dramatically better code than big, vague ones.”*

Problem decomposition is the discipline of breaking a large task into smaller, well-defined sub-tasks that each produce a concrete, testable output.

1.2 The Decomposition Framework

Follow this 4-step approach for any task that would take more than 5 minutes of manual coding:

Step	Action	Example
1. Scope	Define the full outcome you want	"Build a sensor data pipeline with validation, transformation, and reporting"
2. Split	Break into 3–7 independent sub-tasks	"Step 1: CSV reader. Step 2: Schema validator. Step 3: Data transformer. Step 4: Report generator."
3. Sequence	Order by dependencies (what needs to exist first?)	"Reader first (no dependencies) → Validator (needs reader) → Transformer (needs validator) → Reporter (needs all)"
4. Specify	Write a clear prompt for each sub-task	"Create a function read_sensor_csv(filepath: Path) -> pd.DataFrame that reads a CSV, parses timestamps, and validates column names."

1.3 Multi-File Task Decomposition

When a task touches multiple files, decomposition becomes even more critical. Claude Code handles multi-file changes best when you:

Approach	Why It Works
One file at a time	Claude can focus its full context on a single file, producing higher-quality code
Interface first, implementation second	Define function signatures and types first, then implement. This prevents inconsistencies.
Tests alongside code	Ask Claude to generate tests immediately after each function. Catch bugs while context is fresh.
Explicit file references	Use @-references to show Claude exactly which files to modify. Never assume it will find the right file.

Example — Decomposed multi-file prompt sequence:

```
# Prompt 1: Define the interface
> Create @src/models/sensor.py with a Pydantic model SensorReading
> that has fields: timestamp (datetime), sensor_id (str), value (float),
> unit (str). Add validation: value must be between -1000 and 10000.

# Prompt 2: Implement the reader (depends on Prompt 1)
> Create @src/io/csv_reader.py with a function read_sensor_file
> that reads a CSV and returns a list of SensorReading objects.
> Reference the model from @src/models/sensor.py.

# Prompt 3: Add tests (depends on Prompts 1-2)
> Write pytest tests in @tests/test_csv_reader.py for the
> read_sensor_file function. Cover: valid file, empty file,
> malformed rows, missing columns, out-of-range values.
```

☆ **KEY POINT:** Decomposition is not about writing more prompts — it's about writing better prompts. Each sub-task should be small enough that Claude produces excellent code on the first try, reducing the need for iteration.

0:10 – 0:18	SECTION 2: The ReAct Framework — Reason + Act

2. The ReAct Framework — Reason + Act

2.1 What Is ReAct?

ReAct (Reason + Act) is a prompting pattern where you ask Claude to alternate between reasoning about the problem and taking concrete actions. Instead of a single prompt that produces a final answer, you guide Claude through a think-do-review loop.

Traditional Approach	ReAct Approach
One big prompt → One big output	Multiple small prompts → Incremental progress
Hope it gets everything right	Verify each step before moving to the next
Errors compound silently	Errors caught early, before they propagate
Hard to redirect mid-task	Easy to course-correct at each step

2.2 The ReAct Loop in Claude Code

Here's how to implement ReAct in practice:

Phase	What You Do	Claude Code Command
Reason	Ask Claude to analyze the problem and outline an approach	"> Analyze @src/pipeline.py and explain what needs to change to add CSV export."
Act	Ask Claude to implement one specific step	"> Now implement the export_to_csv function in @src/pipeline.py."
Observe	Review the output, run tests, check for issues	"> Run the tests in @tests/test_pipeline.py and show me the results."
Reflect	Ask Claude to evaluate its own work	"> Review the function you just wrote. Are there edge cases we missed?"
Iterate	Refine based on the reflection	"> Good point about empty DataFrames. Add a guard clause for that case."

2.3 ReAct Prompt Templates

Template 1: Analysis First

```
> Before making any changes, analyze @src/core/processor.py and
> @src/core/validator.py. What are the current responsibilities
> of each module? Where is the boundary between them? Are there
```

```
> any overlapping concerns?

# [Review Claude's analysis]

> Good analysis. Now refactor the validation logic out of
> processor.py and consolidate it in validator.py. Keep the
> function signatures backward-compatible.
```

Template 2: Test-Driven ReAct

```
> First, write the test cases for a yield_calculator function
> that takes pass_count and total_count and returns a percentage.
> Include edge cases: zero total, negative values, both zero.

# [Review tests]

> Now implement the yield_calculator function to pass all
> those tests. Reference @tests/test_yield.py for the specs.
```

☆ **KEY POINT:** ReAct turns AI-assisted coding from a “hope it works” approach into a controlled, verifiable process. Each cycle gives you a checkpoint where you can correct course before errors compound.

0:18 – 0:26	SECTION 3: Generation Controls — Temperature & Thinking Budget

3. Generation Controls — Temperature & Thinking Budget

3.1 Understanding Temperature

Temperature controls how “creative” vs. “deterministic” Claude’s responses are. In Claude Code, this affects code generation quality in predictable ways:

Temperature	Behavior	Best For
Low (0.0–0.3)	Highly deterministic. Produces the most likely, conventional code. Less variation between runs.	Boilerplate code, standard patterns, refactoring, format conversions, configuration files
Medium (0.3–0.7)	Balanced. Good trade-off between reliability and creativity. Default for most tasks.	General coding tasks, feature implementation, test generation, documentation
High (0.7–1.0)	More creative and exploratory. May produce unconventional but clever solutions.	Brainstorming approaches, exploring alternative architectures, creative problem-solving

```
# Set temperature via CLI flag:
claude --temperature 0.2

# Or configure in settings:
/config
# Navigate to temperature setting
```

⚠ **WARNING:** In most production coding scenarios, the default temperature works well. Only lower it for highly repetitive or boilerplate tasks, and only raise it when you’re genuinely exploring alternatives. Changing temperature randomly will not improve results.

3.2 Extended Thinking Budget

Extended thinking gives Claude more “time to think” before responding. This is different from temperature — it affects the depth of reasoning, not the creativity of output.

Thinking Budget	When to Use
Default (standard)	Most tasks: function creation, simple refactoring, test writing, documentation

High (extended)	Complex algorithms, architectural decisions, debugging subtle issues, multi-file dependency analysis
Maximum	Very complex reasoning: optimizing algorithms, security audits, designing data pipelines with many edge cases

```
# Enable extended thinking:
claude --thinking-budget high

# Or toggle in session:
/config
# Enable thinking mode

# Check thinking is active:
# Claude will show a [thinking...] indicator before responding
# The thinking process is visible in verbose mode
```

3.3 Combining Controls for Maximum Quality

Task Type	Temperature	Thinking
Boilerplate / config files	Low (0.2)	Default
Standard feature implementation	Default	Default
Complex algorithm design	Default	High
Architecture planning	Default	High
Exploring alternative solutions	Higher (0.7)	High
Debugging a subtle race condition	Low (0.2)	Maximum
Security review / audit	Low (0.2)	Maximum

☆ **KEY POINT:** Extended thinking is your biggest lever for code quality on complex tasks. When you see Claude producing shallow or incorrect solutions for hard problems, enable extended thinking before changing your prompt. It often solves the problem without any prompt changes.

0:26 – 0:34	SECTION 4: Iterative Refinement — From Good to Great

4. Iterative Refinement — From Good to Great

4.1 The Refinement Mindset

First-draft code from Claude is like a first draft of an essay — it captures the structure and intent, but it rarely represents your best work. The engineers who get the most value from AI coding tools are the ones who treat the first output as a starting point, not a final product.

The iterative refinement cycle:

Cycle	Prompt Pattern	What Improves
1. Generate	"Create a function that processes sensor data with validation and error handling."	Initial structure and logic
2. Evaluate	"Review the function you just wrote. What are the potential issues?"	Claude self-identifies weaknesses
3. Harden	"Add edge case handling for: empty input, NaN values, and duplicate timestamps."	Robustness and reliability
4. Optimize	"Profile this function for a dataset of 1M rows. Suggest performance improvements."	Speed and memory efficiency
5. Document	"Add comprehensive docstrings and inline comments explaining the validation logic."	Maintainability

4.2 Effective Feedback Prompts

How you give feedback to Claude determines how well it improves. Be specific about what's wrong and what you want instead:

Vague Feedback (Avoid)	Specific Feedback (Use This)
"This doesn't work"	"The function fails when input_df is empty. Add a guard clause that returns an empty DataFrame."
"Make it faster"	"The nested loop on line 23 is $O(n^2)$. Convert it to a dict lookup for O(n) performance."
"The tests are wrong"	"test_valid_input expects a list but the function now returns a DataFrame. Update the assertion."
"It's not clean enough"	"Extract lines 15–30 into a helper function called _validate_row. The main function should only orchestrate."

"Try again"	"The approach of reading all files into memory won't scale. Use chunked reading with pd.read_csv(chunksize=10000)."

4.3 The /compact + Refine Pattern

For long refinement sessions, use this pattern to prevent context window exhaustion:

```
# Step 1: Initial generation (3-4 prompts)
> Create the sensor pipeline module...
> Add validation...
> Add tests...

# Step 2: Compact to preserve key decisions
/compact

# Step 3: Refine with fresh context
> Now review @src/pipeline/sensor.py and optimize the
> transform_readings function for large datasets. The current
> implementation loads everything into memory.

# Step 4: If needed, compact again before the final pass
/compact

# Step 5: Final polish
> Do a final review of @src/pipeline/sensor.py. Check for:
> type hint completeness, docstring coverage, logging
> consistency, and error handling patterns.
```

☆ **KEY POINT:** The best vibe coders follow the rule of three: generate, refine once for correctness, refine again for quality. Three passes consistently produces code that's ready for code review.

THINK-PAIR-SHARE (2 minutes)

- Think of a recent coding task that took you more than 30 minutes. How would you decompose it?
- What would your first prompt be? What would your refinement prompts look like?
- Share with the person next to you. We'll hear 1–2 examples.

0:34 – 0:42	SECTION 5: Team CLAUDE.md — Scaling Standards Across Engineers

5. Team CLAUDE.md — Scaling Standards Across Engineers

5.1 From Personal to Team Standards

In Module 2, we introduced CLAUDE.md as a personal system prompt. Now we'll build a team-level CLAUDE.md that ensures every engineer on your team gets consistent, standards-compliant code from Claude Code, regardless of their individual prompting skill.

Personal CLAUDE.md	Team CLAUDE.md
Lives in ~/.claude/CLAUDE.md	Lives in project root (version-controlled)
Your individual preferences	Team-agreed standards
Applied to all your projects	Applied only to this project
Style and convenience	Quality, safety, and compliance

5.2 Anatomy of a Team CLAUDE.md

A well-structured team CLAUDE.md has five sections:

```
# CLAUDE.md — TI Sensor Analytics Platform

## 1. Project Context
Python 3.11 data analytics platform for semiconductor
manufacturing sensor data. Runs on Ubuntu 22.04 in Docker.
Used by process engineers and data scientists.

## 2. Code Standards
- Type hints on ALL function signatures (use modern syntax: list[str], not
List[str])
- Google-style docstrings on ALL public functions and classes
- Max function length: 25 lines (extract helpers if longer)
- Use pathlib for all file operations (never os.path)
- Use logging module exclusively (never print() for diagnostics)
- Import order: stdlib, third-party, local (enforced by isort)

## 3. Architecture Rules
- All data models in src/models/ using Pydantic v2
- All I/O operations in src/io/ (readers, writers)
- Business logic in src/core/ (pure functions, no I/O)
- Configuration via environment variables + .env files
- No global mutable state
```

```
## 4. Testing Requirements
- pytest with --strict-markers flag
- Naming: test_{function}_{scenario}_{expected_result}
- Minimum coverage: 85% (checked in CI)
- Use factories for test data (never hardcoded fixtures)
- Mock external dependencies (network, filesystem, clock)

## 5. Safety Guardrails
- NEVER hardcode credentials, API keys, or secrets
- NEVER commit .env files or private keys
- Validate ALL external input before processing
- Use parameterized queries for any database access
- Log all data access operations for audit trail
```

5.3 Advanced CLAUDE.md Techniques

Technique 1: Nested CLAUDE.md for Subdirectories

Place additional CLAUDE.md files in subdirectories to add context-specific rules:

```
# src/io/CLAUDE.md
## I/O Module Standards
- All functions in this directory handle external data sources
- Every function must have timeout parameters with defaults
- Use context managers for file and network handles
- Return empty collections (not None) when no data found
- Log every I/O operation with timing information
```

Technique 2: Negative Rules for Known Pitfalls

Explicitly tell Claude what NOT to do, based on your team's past mistakes:

```
## Anti-Patterns (DO NOT)
- Do NOT use pandas apply() for row-wise operations (use vectorized ops)
- Do NOT catch broad exceptions (except Exception:). Always catch specific
types.
- Do NOT use mutable default arguments (e.g., def func(items=[]))
- Do NOT use string concatenation for SQL queries
- Do NOT store timestamps as strings; always use datetime objects
```

Technique 3: Reference Files for Complex Patterns

Instead of long CLAUDE.md entries, point Claude to example files:

```
## Reference Implementations
- For new validators, follow the pattern in
@src/core/validators/temperature.py
```

```
- For new CLI commands, follow @src/cli/export_command.py as the template
- For new API endpoints, follow @src/api/routes/sensors.py
```

☆ **KEY POINT:** A team CLAUDE.md is a living document. Start with 10–15 rules, then add new rules every time Claude produces code that violates your standards. The CLAUDE.md grows organically into a comprehensive coding standard that's always enforced.

⚠ **WARNING:** Never put sensitive information in CLAUDE.md files. Since CLAUDE.md is sent to Claude's API, it should never contain credentials, internal server addresses, proprietary algorithm details, or anything that shouldn't leave your network. Use generic references instead.

0:42 – 0:45	WRAP-UP & Transition to Module 4

6. Putting It All Together

Let's see how the patterns from this module combine into a complete workflow for a realistic TI task:

6.1 Complete Workflow Example

Task: Add batch processing capability to an existing sensor data pipeline.

🚀 **WORKFLOW DEMONSTRATION**

- Step 1 (Decompose): Break into sub-tasks: batch file discovery, parallel reading, merged validation, aggregated reporting
- Step 2 (ReAct — Reason): "> Analyze @src/pipeline.py. What would need to change to support batch processing of multiple CSV files?"
- Step 3 (ReAct — Act): "> Implement the batch_read_sensor_files function that discovers and reads all CSV files in a directory."
- Step 4 (Observe): Run tests, check output
- Step 5 (Refine): "> The function loads all files into memory. Refactor to use a generator pattern for memory efficiency."
- Step 6 (Team CLAUDE.md): Show how CLAUDE.md standards were automatically applied throughout

---

MODULE 3 RECAP

✅	Problem decomposition: break complex tasks into 3–7 focused sub-tasks. One file at a time, interface before implementation.
✅	ReAct framework: alternate between reasoning and acting. Analyze first, implement second, review third. Catch errors early.
✅	Generation controls: use extended thinking for complex tasks, low temperature for deterministic tasks. Extended thinking is your biggest quality lever.
✅	Iterative refinement: generate, evaluate, harden, optimize, document. Three passes produces code-review-ready output.
✅	Team CLAUDE.md: version-controlled team standards that Claude enforces automatically. Start with 10–15 rules, grow organically.
✅	All patterns work together: decompose the task, use ReAct for each sub-task, enable extended thinking for hard parts, refine iteratively.

QUICK REFERENCE CARD

Keep this reference handy during labs and exercises:

Pattern	When to Use	Key Phrase
Decomposition	Tasks touching 2+ files or >20 lines	"First, let's break this into steps..."
ReAct: Reason	Before any complex implementation	"Analyze @file.py and explain what needs to change"
ReAct: Act	After reviewing the analysis	"Now implement [specific step]"
Extended Thinking	Complex algorithms, architecture, debugging	--thinking-budget high
Low Temperature	Boilerplate, config, format conversion	--temperature 0.2
Iterative Refine	After first-draft code is generated	"Review and identify issues, then fix..."
Team CLAUDE.md	Project setup, onboarding new engineers	Add rules to project root CLAUDE.md

TRANSITION TO MODULE 4

You've now learned both fundamental and advanced prompting patterns for Claude Code. In Module 4, it's time to put everything into practice. You'll work through a hands-on lab where you'll set up a real project with your own CLAUDE.md, decompose a TI-relevant coding task, and build a working solution using all the patterns from Modules 2 and 3.